

Football Analytics AI

Cycle 1 — Football Match Outcome Prediction

Documentation/Findings

1. OVERVIEW

1.1 What Cycle 1 Does

Cycle 1 forms the foundation of the Football Analytics AI project. Its singular goal is to predict the outcome of a Premier League football match — specifically, whether the result will be a Home Win, a Draw, or an Away Win — before the match is played.

This is achieved by training a machine learning model on historical match data spanning 18 complete Premier League seasons (2000/01 through 2017/18). The model learns patterns from features available before a match begins — such as each team's recent form, cumulative goals scored, points per match, and goal difference — and uses those patterns to make predictions about future matches.

The cycle is structured across seven sequential notebooks:

- Exploration — understanding the raw data
- Preprocessing — cleaning and transforming it
- Modelling (Random Split) — training and comparing five models
- Tuning (Random Split) — optimising model settings
- Modelling (Chronological Split) — evaluating under realistic conditions
- Tuning (Chronological Split) — producing the final deployed model
- Explainability — understanding why the model makes each prediction

1.2 The ML Problem Type: Multi-Class Classification

This is a classification problem — the model assigns each match to one of three fixed categories (classes):

- Class 2 — Home Win
- Class 1 — Draw
- Class 0 — Away Win

Because there are three possible labels, this is specifically a multi-class classification problem, distinct from regression (which predicts a continuous number) and binary classification (which predicts one of only two outcomes).

1.3 Why This Problem Matters in Football Analytics

Predicting football outcomes is one of the most widely studied problems in sports data science for several reasons:

- Tactical decision-making: Coaching teams can use model-driven insights to understand which statistical attributes most strongly predict victory.

- Scouting and squad building: Understanding the features that distinguish winning teams from losing ones helps clubs identify what they need.
- Fan engagement and sports media: Probabilistic match predictions are a core product in broadcasting, fantasy sports, and sports journalism.
- Betting markets and financial services: Match outcome prediction underpins a multi-billion-pound industry.

Football is deliberately chosen as a challenging domain. Unlike many classification problems, football outcomes are highly stochastic: a weaker team can beat a stronger team on any given day. A good model must capture probabilistic tendencies, not deterministic rules.

2. AIM AND OBJECTIVES

Aim

To build a robust, interpretable machine learning pipeline that predicts the outcome of Premier League football matches using only pre-match statistical features, evaluated under realistic time-aware conditions.

Objectives

- Conduct thorough exploratory data analysis to understand the structure, distribution, and statistical relationships of the dataset, including class imbalance, collinear features, and temporal trends.
- Design and implement a preprocessing pipeline that transforms raw match data into a clean, model-ready feature matrix — including target variable reconstruction, feature encoding, leakage removal, and column engineering.
- Train and compare multiple machine learning models (Dummy Classifier, Logistic Regression, Random Forest, XGBoost, and LightGBM) on both a standard random split and a chronological split.
- Optimise the best-performing model using hyperparameter tuning via `RandomizedSearchCV` with stratified cross-validation, under strictly chronological conditions.
- Explain model predictions using SHAP to identify which features drive each outcome class, validating that the model has learned sensible football logic.

3. DATASET DESCRIPTION

3.1 Source

The dataset contains historical Premier League match records: 6,840 rows and 40 columns in its raw form. Each row represents a single Premier League match. The dataset spans 18 complete seasons from 2000/01 through 2017/18. Since each season consists of exactly 380 matches (20

teams, each playing 38 games), the expected row count is $18 \times 380 = 6,840$ — confirming the dataset is complete with no missing seasons.

3.2 Target Variable

The target variable is FTR (Full Time Result). After preprocessing, it takes three integer values:

Original Label	Meaning	Encoded Value
H	Home Win	2
D	Draw	1
A	Away Win	0

A critical data quality issue was discovered: the original dataset contained a pre-existing flaw where FTR was binary (Home Win vs Not Home Win). The target was reconstructed from FTHG (Full Time Home Goals) and FTAG (Full Time Away Goals), producing: 3,176 Home Wins (46.4%), 1,751 Draws (25.6%), 1,913 Away Wins (28.0%).

3.3 Key Features

The final processed feature matrix contains 33 features. Key features and their football meaning:

Feature	Description
HTGS / ATGS	Total goals scored by Home/Away team this season
HTGC / ATGC	Total goals conceded by Home/Away team this season
HTGD / ATGD	Goal difference for Home/Away team
HTP / ATP	Points per match for Home/Away team
HM1–HM5	Home team's result in each of last 5 matches (W=3, D=1, L=0)
AM1–AM5	Away team's result in each of last 5 matches
HTFormPts / ATFormPts	Total points from last 5 matches (0–15 range)
DiffPts	Difference in points-per-match (Home HTP – Away ATP)
DiffFormPts	Difference in recent form points
MW	Matchweek (1–38)

HomeTeam / AwayTeam	Label-encoded team identity (44 unique teams)
---------------------	---

4. EXPLORATORY DATA ANALYSIS (EDA)

4.1 What Was Done

- Target variable encoding — FTR converted to numeric at the start of EDA.
- Feature categorisation — columns sorted into pre-match usable vs leakage candidates.
- Target distribution visualisation — bar chart of Home Wins, Draws, Away Wins.
- Missing value analysis — all columns checked for NaN values.
- Feature distribution analysis — histograms for all numeric features.
- Correlation heatmap — Pearson correlations between features and target.
- Box plot analysis — feature distributions per outcome class.
- Outlier detection — IQR method applied to continuous features.
- Temporal stability analysis — match outcome proportions plotted across all 18 seasons.
- Home advantage validation — teams grouped by tenure and home win rates compared.

4.2 Key Findings

Feature Distribution Patterns

Goal accumulation (HTGS/ATGS) — Right-skewed: Statistics reset each season so early-season matches cluster near zero. Only elite teams (e.g., Manchester City) score 80–100+ goals, creating a long right tail.

Points per match (HTP/ATP) — Roughly symmetric: Normalising by matches played removes matchweek bias, creating a near-normal distribution centred around 1.1–1.5 points per game.

Matchweek (MW) — Uniform: Every matchweek from 1 to 38 has the same number of fixtures, producing a perfectly flat histogram.

Point and form differences (DiffPts, DiffFormPts) — Symmetric around zero: Because the schedule is perfectly balanced, positive and negative gaps are equally frequent.

Correlation Heatmap Findings

Finding	Correlation	Implication
HTP and HTGD are near-identical	+0.82	Redundant — teams earn points by scoring more than they concede

DiffFormPts and DiffPts	+0.76	Good teams stay in form; same story told twice
DiffPts vs target	+0.334	Quality gap is the ultimate predictor
HTFormPts and HTP	+0.79	Form is a repeat of overall quality over full season

Outlier Analysis (IQR Method)

The IQR (Interquartile Range) method identifies outliers using fences: Lower Fence = $Q1 - (1.5 \times IQR)$, Upper Fence = $Q3 + (1.5 \times IQR)$. Key findings:

- ~1% of rows flagged for HTGS/ATGS — elite teams scoring 102–105 goals. Legitimate, must not be removed.
- DiffPts flagged values beyond ± 1.77 — extreme top-vs-bottom mismatches. Structurally valid every season.
- DiffFormPts flagged ~8.85% — extreme form divergence. Completely legitimate records.
- Bounded features (Matchweek, form points) — zero outliers.

Conclusion: No rows were dropped or capped. All flagged values represent genuine elite performance or extreme mismatches.

Temporal Stability Analysis

Home advantage is experiencing a weak, noisy decline. The home-win rate averaged ~47.0% in the first three seasons vs ~45.4% in the final three (Spearman correlation: -0.164). Year-to-year variance (~10 percentage points) far outweighs the long-term trend (~2 percentage points). The draw rate remains stable at approximately 25% across all seasons.

The Draw Collapse Problem

Box plots reveal that for every key feature, Draw values sit in the middle of Home Win and Away Win distributions. This means the model cannot draw a clean boundary for draws — they are sandwiched between winning distributions at every feature dimension. This is the structural draw collapse problem that persists across all models.

4.3 Conclusion of EDA

Problem	Impact	Action
FTR was binary (H vs not-H)	Cannot train 3-class model	Reconstruct from FTHG/FTAG

FTHG/FTAG present	Data leakage — scoreline reveals result	Drop after FTR reconstruction
String form columns	Models require numeric input	Encode W=3, D=1, L=0
String team names	Models require numeric input	Label encode
Season is metadata	Would teach wrong year-specific patterns	Use for splitting only, then drop
Class imbalance 46/25/29%	Model biased toward Home Wins	Use class_weight='balanced'
Draw overlap in feature space	Structurally hard to predict	Acknowledged as fundamental limitation

5. PREPROCESSING

5.1 Pipeline Summary

Step	Action	Before	After
1	Load raw data	—	(6840, 40)
2	Drop useless columns	(6840, 40)	(6840, 37)
3	Reconstruct FTR (H/D/A)	Binary H/NH	3-class H/D/A
4	Drop FTHG/FTAG (leakage)	(6840, 37)	(6840, 35)
5	Encode form columns	W/D/L/M strings	3/1/0/0 integers
6	Encode team names	String names	Integer labels (0–43)
7	Extract Season, drop Date	Date string	Season integer
8	Convert MW to int	float64	int64
9	Encode target FTR	H/D/A strings	2/1/0 integers
10	Final validation	—	(6840, 35), 0 missing

11	Save to processed/	—	premier_league_matches_processed.csv
----	--------------------	---	--------------------------------------

5.2 Key Preprocessing Decisions Explained

Reconstructing FTR — The Most Critical Step

The original FTR column only recorded 'H' or 'NH' — a binary field that cannot support three-class prediction. Reconstruction used objective goal counts: $FTHG > FTAG \rightarrow H$, $FTHG == FTAG \rightarrow D$, $FTHG < FTAG \rightarrow A$. This produces exactly 6,840 rows with no data loss.

Data Leakage — Dropping FTHG and FTAG

Data leakage occurs when information unavailable at prediction time enters the training data, allowing the model to 'cheat.' FTHG and FTAG are the match scoreline — the answer to what we are predicting. If kept, the model trivially learns 'If $FTHG > FTAG$, predict Home Win,' producing near-perfect training accuracy but zero real-world utility. They were dropped immediately after FTR reconstruction.

Encoding Form Columns (W/D/L/M $\rightarrow$ 3/1/0/0)

The encoding uses actual football point values: W=3 (a win earns 3 points), D=1 (draw earns 1 point), L=0 (loss earns 0 points), M=0 (no prior match — neutral). This preserves real-world meaning: a win is worth three times a draw. The M=0 strategy zero-fills early-season NaN values, accurately representing the absence of match history without fabricating statistics.

Missing Value Strategy

Standard mean imputation (SimpleImputer) would be incorrect for early-season rolling statistics. Filling Matchday 1 values with a mid-season average would distort the model's understanding of early-season form. Instead, NaN values are zero-filled — representing the genuine absence of prior match history.

5.3 Output Dataset

Final processed file: data/processed/premier_league_matches_processed.csv

- 6,840 rows (no rows lost)
- 35 columns (33 features + FTR target + Season identifier)
- 0 missing values
- All columns numeric

6. MODELLING

6.1 Models Used

Dummy Classifier

Predicts Home Win (the most common class) for every match, ignoring all features. Achieves 46.35% accuracy purely from class distribution. Any real model must significantly exceed this. AUC-ROC = 0.5 (random).

Logistic Regression

Learns linear weights that combine feature values to estimate the probability of each outcome. Draws straight lines between classes in feature space — it can only learn linear relationships. Used with `class_weight='balanced'` to counteract the 46%/25%/29% imbalance. Confirms whether the features carry genuine predictive signal at all.

Random Forest

Builds 100 independent decision trees, each on a random subset of data with random feature selection, combining predictions by majority vote. Addresses a single tree's tendency to overfit by averaging across many trees. Captures non-linear interactions between features that Logistic Regression cannot.

XGBoost (Extreme Gradient Boosting)

Builds decision trees sequentially — each new tree corrects the errors of all previous trees. Unlike Random Forest's democratic voting, XGBoost focuses learning on hard-to-predict cases. This sequential error-correction makes it particularly attentive to draws (the hardest class), and it is one of the strongest algorithms for structured tabular data.

LightGBM

A gradient boosting model that grows trees leaf-wise (best-first) rather than level-wise. It always expands the single node providing the greatest error reduction, making it faster and often more accurate than XGBoost without tuning. Its aggressive growth strategy can risk overfitting — as the chronological evaluation later reveals.

6.2 Training Approach — Random vs Chronological Split

Two fundamentally different splitting strategies are used:

Random Split (`train_test_split`): Rows are randomly shuffled before splitting 80/20. A 2018 match might end up in training while a 2005 match ends up in testing. `random_state=42` ensures the same split every run, guaranteeing reproducibility.

Chronological Split: Data sorted by Season in ascending order. The earliest 80% of matches (approx. seasons 2000–2014) form the training set. The latest 20% (approx. seasons 2015–2018) form the test set. The model strictly learns from the past and is evaluated on the future — faithfully replicating real deployment conditions.

6.3 Evaluation Metrics Explained

Metric	Definition	Why It Matters Here
Accuracy	% of all predictions that are correct	Misleading due to imbalance; reported for completeness
Precision	Of predicted class X, what % were correct?	Tells us how confident predictions are
Recall	Of actual class X, what % did the model find?	Tells us how many real cases are caught
F1-Score	Harmonic mean of precision and recall	Balances both — penalises extreme imbalances
AUC-ROC	Area under ROC curve; 0.5=random, 1.0=perfect	Threshold-independent discrimination measure

6.4 Results Summary

Model	Accuracy	vs Dummy	HW Recall	Draw Recall	AW Recall
Dummy Classifier	46.35%	—	1.00	0.00	0.00
Logistic Regression	49.85%	+3.51pp	0.57	0.25	0.59
Random Forest	51.39%	+5.04pp	0.80	0.10	~0.42
XGBoost	50.95%	+4.61pp	~0.72	0.20	~0.48
LightGBM	54.02%	+7.68pp	0.79	0.18	0.44

Key observation: Every model beats the Dummy by 3.51–7.68 percentage points, confirming the features carry real predictive signal. LightGBM achieves the highest accuracy. Draw recall remains extremely low (0.10–0.25) across all models — a structural limitation of the dataset, not a modelling failure.

7. HYPERPARAMETER TUNING

7.1 What Are Hyperparameters?

Every machine learning model has two types of parameters. Model parameters are values the model learns from data during training (e.g., decision tree split thresholds). Hyperparameters are settings chosen before training begins that control how the model learns — the model cannot discover these itself.

Analogy: Think of a model as an oven. The data is the ingredients, the model is the oven, and hyperparameters are the temperature, fan speed, and timer settings. Even with perfect ingredients, wrong oven settings produce a bad result.

7.2 RandomizedSearchCV

Rather than testing every possible combination of hyperparameters (GridSearchCV), RandomizedSearchCV randomly samples 50 combinations from defined ranges and evaluates each using 5-fold Stratified Cross-Validation. Total model fits per algorithm: $50 \times 5 = 250$.

Why not GridSearchCV? The XGBoost grid contains 3,888 possible combinations. With 5-fold CV, this requires $3,888 \times 5 = 19,440$ model fits — computationally prohibitive.

RandomizedSearchCV finds near-optimal solutions at a fraction of the cost.

StratifiedKFold ensures each fold preserves the same class proportions (46% HW / 25% D / 29% AW), preventing biased evaluation from imbalanced folds.

7.3 Tuning Results

Model	CV AUC	Test Accuracy	Best Key Params
Tuned XGBoost (Random)	52.49%	52.78%	n_est=500, lr=0.01, depth=4
Tuned LightGBM (Random)	52.32%	53.29%	n_est=200, lr=0.01, depth=3
Tuned XGBoost (Chrono)	53.02%	52.85% ← DEPLOYED	n_est=500, lr=0.01, depth=4

Tuned LightGBM on the random split (54.61%) edges tuned XGBoost (52.78%). However, LightGBM's large chronological performance drop reveals over-reliance on temporal artefacts. Tuned XGBoost proves most robust under strict chronological conditions, becoming the deployed model.

8. CHRONOLOGICAL VS RANDOM SPLIT

8.1 Data Leakage Explained

Data leakage occurs when information from the future accidentally enters the training process, giving the model an unfair advantage it would not have in real-world deployment.

Concrete example: In a random split, some 2016/17 matches might end up in the training set. When the model encounters similar situations in the test set (from the same season), it recognises memorised patterns rather than generalising. This is analogous to giving a student the exam questions the night before and calling it a genuine assessment.

8.2 Impact Quantified

Model	Chronological	Random	Drop (Δ)
Dummy	44.88%	46.35%	-1.47pp
Logistic Regression	48.76%	49.85%	-1.09pp
Random Forest	51.17%	51.39%	-0.22pp
XGBoost	48.17%	50.95%	-2.78pp
LightGBM	49.85%	54.02%	-4.17pp

Key pattern: The more powerful the model, the larger the performance drop when switching to the chronological split. LightGBM drops 4.17pp — its aggressive leaf-wise learning makes it highly effective at memorising temporal patterns that constitute leakage. Random Forest is the most stable (-0.22pp), suggesting ensemble averaging provides natural resistance to overfitting specific time periods.

8.3 Deployed Model

Tuned XGBoost (Chronological) is the deployed model, achieving 52.85% accuracy on the held-out future seasons. It is saved as `models/cycle1/cycle1_xgb_best.pkl` alongside `cycle1_feature_cols.pkl` (33 column names in correct order).

9. EVALUATION

9.1 Confusion Matrix

The confusion matrix is a 3×3 grid showing how predictions relate to actual outcomes. Diagonal cells are correct predictions; off-diagonal cells are errors. For this model: Home Wins are well identified (strong diagonal entry); Draws are frequently misclassified as Home Wins or Away Wins (weak diagonal, spread across columns); Away Wins are moderately identified.

9.2 ROC Curve

The ROC curve plots True Positive Rate against False Positive Rate as the decision threshold is swept from 0 to 1. AUC = 1.0 is perfect; AUC = 0.5 is random guessing. Home Win has the highest AUC (most separable); Draw has the lowest (least separable). The chronological model shows clear separation from the diagonal, confirming genuine discriminative ability.

9.3 Strengths and Weaknesses

Strengths:

- Consistent improvement over baseline: All five models beat the Dummy Classifier.
- Strong Home Win prediction: Recall reaches 0.86–0.87 in tuned models.
- Realistic evaluation framework: Chronological split ensures 52.85% reflects true generalisation.
- Robustness: CV accuracy (53.02%) closely matches test accuracy (52.85%) — no overfitting.

Weaknesses:

- Draw prediction consistently fails: Recall = 0.05–0.20 across all models.
- Class imbalance persists despite balanced weighting.
- Feature redundancy: DiffPts, HTP, and HTGD carry overlapping information.
- No in-game data: possession, shots on target, and match momentum are absent.

10. EXPLAINABILITY (SHAP)

10.1 What is SHAP?

SHAP (SHapley Additive exPlanations) is a method rooted in cooperative game theory. It calculates, for each feature in each prediction, how much that feature pushed the prediction towards or away from a class. SHAP values are mathematically guaranteed to sum exactly to the model's output — they are internally consistent and fair across all features.

A positive SHAP value means the feature pushed the prediction toward that class. A negative SHAP value means it pushed the prediction away from that class.

10.2 Global Feature Importance

Rank	Feature	Interpretation
1	DiffPts	Dominant predictor — quality gap between teams
2	AM1 (Away Match 1)	Away team's most recent result
3	HM1 (Home Match 1)	Home team's most recent result
4	ATGD	Away team's cumulative goal difference
5	HTGD	Home team's cumulative goal difference
6+	HomeTeam, AwayTeam	Moderate — team-specific tendencies learned
Bottom	HM4, HM5, AM4, AM5	Near-zero — older match history loses predictive power

10.3 SHAP Findings by Class

Home Win: High DiffPts strongly pushes predictions toward Home Win. Strong recent home form (HM1=3) increases probability. Strong away form (AM1=3) reduces it.

Away Win: Low/negative DiffPts pushes toward Away Win. Strong away form (AM1=3) is the key driver.

Draw: No feature exhibits a strong consistent directional effect. SHAP magnitudes are small and scattered — confirming the draw class has no distinguishing feature signal.

11. DEPLOYMENT

11.1 Saved Artefacts

- cycle1_xgb_best.pkl — Tuned XGBoost classifier (chronological split)
- cycle1_feature_cols.pkl — 33 feature column names in correct order

11.2 FastAPI

FastAPI serves the model as a REST API. A POST request containing pre-match features (as JSON) is received, validated, aligned to the saved column order, passed through the XGBoost model, and the predicted class (Home Win / Draw / Away Win) plus probabilities are returned as a JSON response.

11.3 Streamlit Dashboard

The Streamlit dashboard allows users to: (1) select home and away teams from dropdowns; (2) enter matchweek and season; (3) click Predict to send features to the FastAPI endpoint; (4) view the predicted outcome, probability bar chart, and a SHAP waterfall plot explaining which features drove the prediction.

11.4 Full Pipeline

```
User selects teams & matchweek
→ Dashboard assembles 33 features
→ POST request → FastAPI endpoint
→ Load pkl files → Align features
→ XGBoost.predict_proba(input)
→ Return predicted class + probabilities
→ Streamlit renders outcome + SHAP waterfall
```

12. CONCLUSION

Cycle 1 successfully delivered a complete end-to-end machine learning pipeline. A raw 40-column dataset was diagnosed, cleaned, and transformed into a model-ready 33-feature matrix. Five models were systematically trained and compared, progressing from 46.35% to 52.85%. The critical distinction between random and chronological evaluation was quantified, proving naive splitting inflates performance by up to 4.17 percentage points. SHAP analysis validated that the model reasons from coherent football logic centred on team quality difference and recent form.

Key Strengths

- Methodological rigour: Explicit separation of random-split (comparison) and chronological-split (deployment) evaluation.
- Leakage prevention: FTHG/FTAG dropped immediately after reconstruction; Season excluded from feature matrix.
- Explainability: SHAP validates reasoning at the individual prediction level.
- Reproducibility: random_state=42 and fixed column ordering.

Limitations

- Draw recall = 0.05–0.20 across all models — a structural data limitation.
- No in-game features (possession, shots, red cards).
- Label encoding implies false team ordering for linear models.
- No external context (injuries, managerial changes, weather).

13. FUTURE IMPROVEMENTS

- Feature engineering for draw-specific signals: absolute DiffPts, head-to-head draw history, season-phase indicators.
- Probabilistic calibration: Platt scaling or isotonic regression to calibrate output probabilities.
- Ensemble combining chronological XGBoost and Random Forest (most temporally stable models).
- Incorporate betting market odds as a feature — implicitly captures expert knowledge about injuries and team news.
- One-hot encoding for team names — removes false numerical ordering, benefiting Logistic Regression.
- Time-decay weighting — give recent seasons more weight during training.